Economics S5040: Dynamic Stochastic Modeling
Lecture 5 — Matrices, Expectations, and Monte Carlo Methods

Eric Fisher

Fall 2026

1 Why We Need a Little Linear Algebra

Much of modern macroeconomics describes an economy that can be in one of several states. For example, a worker might be employed or unemployed, or the economy might be in an expansion or a recession.

To study how probabilities move across states, we need a convenient way to organize numbers and perform weighted sums. Vectors and matrices provide exactly that language.

Our goal today is modest. We need to know how to

- read vectors and matrices;
- multiply a vector by a matrix;
- multiply two matrices;
- interpret these operations as weighted sums;
- use the same idea to understand expectations and Monte Carlo approximation.

We will not study determinants, inverses, eigenvalues, or general linear algebra today.

2 Vectors and Matrices

A vector is an ordered list of numbers. For example,

$$x = \begin{pmatrix} 2 \\ 5 \end{pmatrix}$$

is a 2×1 column vector, while

$$p = \begin{pmatrix} 0.8 & 0.2 \end{pmatrix}$$

is a 1×2 row vector.

A matrix is a rectangular array of numbers. For example,

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

is a 2×2 matrix.

The first index identifies the row and the second identifies the column. Thus, if

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix},$$

then a_{21} is the entry in row 2, column 1.

In Python, we can create these objects with NumPy:

```
import numpy as np

x = np.array([2, 5])
p = np.array([0.8, 0.2])
A = np.array([[1, 2],
              [3, 4]])
```

3 Matrix Multiplication

The central rule is simple:

> To obtain an entry in a matrix product, multiply the entries in a row of the first matrix by the corresponding entries in a column of the second matrix, and add.

For example,

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 6 \end{pmatrix}.$$

Then

$$Ax = \begin{pmatrix} 1(5) + 2(6) \\ 3(5) + 4(6) \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}.$$

For two matrices,

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix},$$

we obtain

$$AB = \begin{pmatrix} 1(5) + 2(7) & 1(6) + 2(8) \\ 3(5) + 4(7) & 3(6) + 4(8) \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

The dimensions must be compatible. If A is $m \times n$ and B is $n \times k$, then AB is $m \times k$.

In NumPy, matrix multiplication uses the symbol `@`:

```
A @ x
A @ B
```

Do not confuse this with

```
A * B
```

which multiplies entries element by element.

4 An Economic Example

Suppose that today 80 percent of workers are employed and 20 percent are unemployed. Write the distribution as

$$\psi_0 = \begin{pmatrix} 0.8 & 0.2 \end{pmatrix}.$$

Suppose further that an employed worker has a 90 percent chance of being employed tomorrow, while an unemployed worker has a 40 percent chance of being employed tomorrow. Organize these probabilities as

$$P = \begin{pmatrix} 0.9 & 0.1 \\ 0.4 & 0.6 \end{pmatrix}.$$

Each row sums to one. The first row describes what can happen tomorrow to someone who is employed today; the second row describes what can happen to someone who is unemployed today.

Tomorrow's distribution is

$$\psi_1 = \psi_0 P.$$

The fraction employed tomorrow is therefore

$$(0.8)(0.9) + (0.2)(0.4) = 0.80,$$

and the fraction unemployed tomorrow is

$$(0.8)(0.1) + (0.2)(0.6) = 0.20.$$

Thus

$$\psi_1 = \begin{pmatrix} 0.8 & 0.2 \end{pmatrix}.$$

This particular distribution happens not to change. Next lecture we will give such a distribution a name and study when it exists.

In Python:

```
psi = np.array([0.8, 0.2])
P = np.array([[0.9, 0.1],
              [0.4, 0.6]])

psi_next = psi @ P
print(psi_next)
```

The important economic point is that matrix multiplication is performing a collection of weighted averages.

5 Expectations Are Weighted Averages

Suppose a random variable X can take values $x_1, \ldots, x_n$ with probabilities $p_1, \ldots, p_n$. Then

$$E[X] = \sum_{i=1}^{n} p_i x_i.$$

More generally, for a function g,

$$E[g(X)] = \sum_{i=1}^{n} p_i g(x_i).$$

For example, suppose next year's income is

$$y = 100 \quad \text{with probability } 0.8$$

and

$$y = 20 \quad \text{with probability } 0.2.$$

Then expected income is

$$E[y] = (0.8)(100) + (0.2)(20) = 84.$$

If X is continuous with density f, the corresponding expression is

$$E[g(X)] = \int g(x)f(x)\,dx.$$

The discrete sum and the continuous integral express the same idea: an expectation is an average in which outcomes are weighted by their probabilities.

6 The Law of Large Numbers

Suppose we draw independent observations

$$X_1, X_2, \ldots, X_N$$

from the same probability distribution.

The Law of Large Numbers tells us that, under standard conditions,

$$\frac{1}{N}\sum_{i=1}^{N} g(X_i) \longrightarrow E[g(X)]$$

as N becomes large.

This result gives us a computational method. If an expectation is difficult to calculate directly, we can simulate many observations and take their average.

This is the basic idea of **Monte Carlo approximation**.

7 Monte Carlo Integration

Consider

$$I = \int_0^1 x^2\,dx.$$

We know analytically that

$$I = \frac{1}{3}.$$

Now let X be uniformly distributed on $[0, 1]$. Since the uniform density is one on this interval,

$$E[X^2] = \int_0^1 x^2\,dx = \frac{1}{3}.$$

Therefore we can approximate the integral by drawing

$$X_1, \ldots, X_N \sim U[0, 1]$$

and computing

$$\widehat{I}_N = \frac{1}{N}\sum_{i=1}^{N} X_i^2.$$

In Python:

```
import numpy as np

rng = np.random.default_rng(1234)

N = 10_000
x = rng.uniform(0, 1, size=N)
I_hat = np.mean(x**2)

print("Monte Carlo estimate:", I_hat)
print("Exact answer:", 1/3)
```

Try changing N to

$$10, \quad 100, \quad 1{,}000, \quad 10{,}000, \quad 100{,}000.$$

The estimate does not move monotonically toward $1/3$, but its typical error becomes smaller as the number of draws grows.

We can see this visually:

```
import matplotlib.pyplot as plt

N = 10_000
x = rng.uniform(0, 1, size=N)
running_estimate = np.cumsum(x**2) / np.arange(1, N + 1)

plt.figure(figsize=(7, 4))
plt.plot(running_estimate, label="Monte Carlo estimate")
plt.axhline(1/3, linestyle="--", label="Exact value")
plt.xlabel("Number of draws")
plt.ylabel("Estimate")
plt.legend()
plt.show()
```

8 Why Economists Use Monte Carlo Methods

In simple examples we can often calculate an expectation exactly. In more complicated models, however, expectations can be difficult or impossible to evaluate analytically.

Simulation gives us another route:

$$\boxed{\text{probability model} \longrightarrow \text{random draws} \longrightarrow \text{sample average} \approx \text{expectation}}$$

We will use this idea repeatedly. In dynamic models, agents often care about expected future values. Monte Carlo methods allow us to approximate those expectations numerically.

9 Looking Ahead: Markov Chains

Return to our employment example:

$$\psi_{t+1} = \psi_t P.$$

Next lecture we will interpret P as a **Markov transition matrix**. We will ask:

- How do we calculate the distribution two, ten, or one hundred periods ahead?
- Can we simulate an individual's employment history?
- Does the distribution of workers eventually settle down?
- If it does, can we calculate the long-run distribution without simulating thousands of periods?

The matrix multiplication we learned today will provide the basic language for answering these questions.